Project Plan Documentation

Project Name:- FarmVerse Precision Agriculture Management Platform

1. Problem Statement

Traditional farming methods rely heavily on manual record-keeping, making it difficult for farmers to efficiently manage crop information, irrigation schedules, fertilizer usage, expenses, harvest records, and weather updates. Paper-based records are often inaccurate, time-consuming to maintain, and difficult to analyze. Farmers also lack a centralized system to monitor farming activities and generate reports for better decision-making.

FarmVerse addresses these challenges by providing a centralized web-based agriculture management platform that digitizes farm operations, improves record management, and supports informed agricultural decisions.

2. Proposed Architecture

The system follows a **Three-Tier Architecture**.

Presentation Layer :-

- HTML
- CSS
- JavaScript
- Bootstrap

Responsible for user interaction and displaying information.

Business Logic Layer :-

- Java
- Spring Boot
- Spring MVC
- Spring Security
- REST APIs

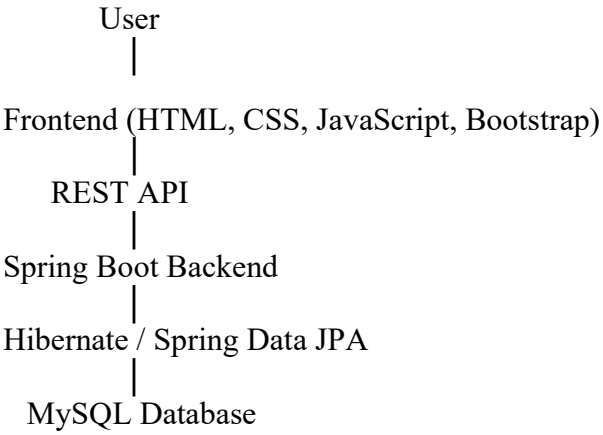
Responsible for authentication, business logic, validations, and communication between frontend and database.

Data Layer :-

- MySQL
- Spring Data JPA (Hibernate)

Stores all application data such as users, farms, crops, irrigation schedules, expenses, and reports.

Architecture Flow



3. Requirement Gathering :-

Functional Requirements

- User Registration and Login
- Role-Based Authentication
- Farm Management
- Crop Management
- Irrigation Scheduling
- Fertilizer & Pesticide Tracking
- Expense Management
- Harvest Management
- Weather Information
- Notifications
- Reports Generation
- Admin Dashboard

Non-Functional Requirements

- Security
- High Performance
- Reliability
- Scalability
- Easy Maintenance
- Responsive UI
- Data Integrity
- Cross-Browser Compatibility

4. Project Modules

4.1 User Management Module

This module manages user registration, authentication, and account management. It ensures secure access to the system for both farmers and administrators.

Main Features:

- Registration
- Login
- Profile Management
- Password Management
- Logout

4.2 Farm Management Module

This module allows farmers to manage their farm information and maintain farm records.

Main Features:

- Add Farm
- Update Farm
- Delete Farm
- View Farm Details

4.3 Crop Management Module

This module helps farmers manage crop information from sowing to harvesting.

Main Features:

- Add Crop
- Update Crop
- Track Crop Status
- Harvest Information

4.4 Irrigation Management Module

This module helps farmers schedule and monitor irrigation activities.

Main Features:

- Schedule Irrigation
- Water Quantity
- Irrigation History

4.5 Fertilizer & Pesticide Management Module

This module records fertilizer and pesticide usage for crops.

Main Features:

- Add Fertilizer
- Add Pesticide
- Usage History
- Cost Tracking

4.6 Expense Management Module

This module helps farmers record and monitor farming expenses.

Main Features:

- Seed Cost
- Labour Cost
- Machinery Cost
- Total Expense Calculation

4.7 Harvest Management Module

This module manages harvest details and calculates farm income.

Main Features:

- Harvest Quantity
- Selling Price
- Income Calculation
- Profit/Loss Analysis

4.8 Weather Module

This module provides weather information to support farming decisions.

Main Features:

- Temperature
- Rainfall
- Humidity
- Weather Alerts

4.9 Reports Module

This module generates reports to help farmers analyze farm activities and performance.

Main Features:

- Crop Reports
- Irrigation Reports
- Expense Reports
- Harvest Reports
- Profit/Loss Reports

5. Technology Stack

Frontend

- HTML5
- CSS3
- JavaScript
- Bootstrap

Backend

- Java
- Spring Boot
- Spring MVC
- Spring Security
- REST APIs
- Hibernate
- Spring Data JPA

Database

- MySQL

6. Database Entities and Attributes

User = { UserID (PK), Name, Email, Password, Phone, Role }

Farm = { FarmID (PK), UserID (FK), FarmName, Location, Area, SoilType }

Crop = { CropID (PK), FarmID (FK), CropName, CropType, SowingDate, HarvestDate,Status }

Irrigation = { IrrigationID (PK), CropID (FK), IrrigationDate, WaterQuantity, Method }

Fertilizer = { FertilizerID (PK), CropID (FK), FertilizerName, Quantity, Cost, Date }

Pesticide = { PesticideID (PK), CropID (FK), PesticideName, Quantity, Cost, Date }

Expense = { ExpenseID (PK), FarmID (FK), ExpenseType, Amount, Date }

Harvest = { HarvestID (PK), CropID (FK), Quantity, SellingPrice, TotalIncome, Date }

Weather = { WeatherID (PK), Temperature, Humidity, Rainfall, ForecastDate }

Notification = { NotificationID (PK), UserID (FK), Message, Date, Status }

7. Functional Requirements

- User Registration
- Secure Login
- Role-Based Access

- Farm Management
- Crop Management
- Irrigation Scheduling
- Fertilizer Management
- Expense Tracking
- Harvest Management
- Weather Updates
- Notifications
- Reports Generation

8. Non-Functional Requirements

- Secure Authentication
- Fast Response Time
- Responsive Interface
- Data Backup
- High Availability
- Reliability
- Scalability
- Easy Maintenance
- Data Accuracy

9. Week-wise Internship Plan (6 Weeks)

Week	Frontend (FE)	Backend (BE)	Database Administrator (DA)
Week 1	Requirement Analysis, UI Wireframes	Spring Boot Project Setup	ER Diagram and Database Design
Week 2	Login, Registration UI	Authentication APIs	Create MySQL Tables
Week 3	Farm & Crop Pages	Farm & Crop APIs	Relationships and Sample Data
Week 4	Irrigation & Dashboard UI	Irrigation, Reports APIs	Query Optimization
Week 5	Responsive Design, API Integration	Testing, Security, Bug Fixes	Backup and Performance Tuning
Week 6	Final UI, Documentation	Deployment and Final Integration	Database Verification and Documentation

10. Team Responsibilities

Frontend Developer (FE)

- UI Design
- Responsive Web Pages
- API Integration
- Client-side Validation
- Dashboard Development

Backend Developer (BE)

- REST API Development
- Business Logic
- Authentication
- Authorization
- Spring Security
- Exception Handling

Database Administrator (DA)

- Database Design
- Table Creation
- Relationships
- Query Optimization
- Backup
- Data Integrity

11. Testing

Unit Testing

Individual modules are tested independently.

Examples:

- Login Module
- Registration Module
- Farm Module
- Crop Module
- Irrigation Module
- Expense Module

Integration Testing

- User + Farm Module
- Crop + Irrigation Module
- Expense + Harvest Module

System Testing

Entire application is tested as a complete system.

User Acceptance Testing (UAT)

Farmers and administrators verify that all functionalities meet the project requirements.

12. Expected Output

The final system will provide:

- Secure Farmer and Admin Login
- Farm Management Dashboard
- Crop Management System
- Irrigation Scheduling
- Fertilizer & Pesticide Tracking
- Expense Tracking
- Harvest Management
- Profit/Loss Calculation
- Weather Information
- Notifications and Alerts
- Reports and Analytics Dashboard
- Responsive and User-Friendly Interface
- Reliable MySQL Database Management
- Secure Java Spring Boot Backend

The completed application will help farmers manage agricultural activities efficiently, reduce manual record-keeping, improve productivity, and support better decision-making through a centralized digital platform.